

OTEC
PRECISION FINISHING SOLUTIONS

HOW TO MAKE DJANGO INSECURE

Swiss Python Summit 2026 · Rapperswil

OTEC
PRECISION FINISHING SOLUTIONS

$ whoami

```
➜ whoami
```


Traveller


Handball Referee


Board Game Geek


Pythonista

WHAT DOES THE COMPANY DO?



Grinding Machines



Electro Finishing Machines

OTEC
PRECISION FINISHING SOLUTIONS

WHY THIS TALK

DJANGO SHIPS WITH SECURITY ON

- On by default, in `startproject`
- Most of it before you even know the attack exists
- Each protection can be switched off — with one line

WHO SWITCHES IT OFF?

- **We do**
- **Our assistants do**
- More code, less attention per line

> Today: switch it off, see what Django does under the hood — and the prompt that switched it off.

OTEC
PRECISION FINISHING SOLUTIONS

OWASP TOP 10 — 2025

CATEGORY	2021	2025	TODAY
Broken Access Control	1	**1**	invoices, profile form
Security Misconfiguration	5	**2**	`settings.py`
Software Supply Chain Failures	6	**3**	
Cryptographic Failures	2	**4**	password hashing
Injection	3	**5**	search, bio
Insecure Design	4	**6**	

OTEC
PRECISION FINISHING SOLUTIONS

LET'S GO

SWITCHING IT OFF

Five protections, five prompts, five fixes

INJECTION — THE INSECURE WAY

```
def search(request):
    q = request.GET.get("q", "")
    products = Product.objects.raw(
        f"SELECT * FROM shop_product WHERE name LIKE '%{q}%'"
    )
    return render(request, "results.html", {"products": products})
```

```
/search/?q=' OR '1'='1
```

> **User input became SQL.** The search now returns every row — including the ones that were never meant to be listed.

OTEC
PRECISION FINISHING SOLUTIONS

INJECTION — HOW DJANGO PREVENTS IT

```python
def search(request):
    q = request.GET.get("q", "")
    products = Product.objects.filter(name__icontains=q)
```

```python
# django/db/models/sql/compiler.py (simplified)
sql, params = self.as_sql()
cursor.execute(sql, params)
```

```python
# If you really need raw SQL: use placeholders
Product.objects.raw("SELECT * FROM shop_product WHERE name LIKE %s", [f"%{q}%"])
```

MAKE THE MACHINE DO IT

`ruff check --select S` → `S608` string-built SQL, `S610` `.extra()`, `S611` `RawSQL`.

OTEC
PRECISION FINISHING SOLUTIONS

XSS — THE INSECURE WAY

PROMPT

My HTML from the database renders as text instead of markup. How do I fix that?

ASSISTANT

Django escapes template variables by default. Use the `|safe` filter.

```
<div class="bio">{{ profile.bio|safe }}</div>
```

```
<!-- what somebody puts in their bio -->
<script>alert(document.cookie)</script>
```

That answer is correct. It solves the problem that was described — and every visitor of that profile now runs the author's JavaScript.

XSS — HOW DJANGO PREVENTS IT

```
# django/utils/html.py (simplified)
_html_escapes = {ord("&"): "&amp;", ord("<"): "&lt;", ord(">"): "&gt;",
                 ord('"'): "&quot;", ord("'"): "&#x27;"}

def escape(text):
    return SafeString(str(text).translate(_html_escapes))
```

```
import nh3

class Profile(models.Model):
    def save(self, *args, **kwargs):
        self.bio = nh3.clean(self.bio, tags={"b", "i", "em", "strong", "p", "br"})
        super().save(*args, **kwargs)
```

MAKE THE MACHINE DO IT

`S308` flags `mark_safe`. Sanitise on the way **in** — then `|safe` is a statement about checked data, not a hope.

OTEC
PRECISION FINISHING SOLUTIONS

CSRF — THE INSECURE WAY

> PROMPT
>
> The React frontend gets "403 CSRF verification failed" on POST /transfer/. Fix it.

```python
@csrf_exempt
@login_required
def transfer_money(request):
    transfer(request.user, to=request.POST["to"], amount=request.POST["amount"])
    return JsonResponse({"status": "ok"})
```

```html
<!-- on any other website the user happens to visit -->
<form action="https://bank.example.com/transfer/" method="post">
  <input type="hidden" name="to" value="someone-else">
  <input type="hidden" name="amount" value="10000">
</form>
<script>document.forms[0].submit()</script>
```

> The browser sends the **session cookie** along. The view cannot tell who built the form.

OTEC
PRECISION FINISHING SOLUTIONS

CSRF — HOW DJANGO PREVENTS IT

```python
# django/middleware/csrf.py (simplified)
class CsrfViewMiddleware(MiddlewareMixin):
    def process_view(self, request, callback, callback_args, callback_kwargs):
        if getattr(callback, "csrf_exempt", False):
            return None                                  # ← what the decorator does
        if request.method in ("GET", "HEAD", "OPTIONS", "TRACE"):
            return self._accept(request)
        # check Origin / Referer, then the token
        ...
```

```javascript
// The actual fix for the 403: send the token
fetch("/transfer/", {
  method: "POST",
  headers: { "X-CSRFToken": getCookie("csrftoken") },
  body: formData,
})
```

OTEC
PRECISION FINISHING SOLUTIONS

ACCESS CONTROL — THE INSECURE WAY

> PROMPT
>
> Add a detail page for invoices at /invoices/<id>/ and a form so users can edit their profile.

```
def invoice_detail(request, invoice_id):
    invoice = Invoice.objects.get(id=invoice_id)
    return render(request, "invoice.html", {"invoice": invoice})

class ProfileForm(forms.ModelForm):
    class Meta:
        model = User
        fields = "__all__"              # includes is_staff, is_superuser
```

```
/invoices/41/  → your invoice
/invoices/42/  → somebody else's invoice
```

It reads your **data model**. It cannot read your **threat model**.

OTEC
PRECISION FINISHING SOLUTIONS

ACCESS CONTROL — HOW DJANGO HELPS

```python
@login_required
def invoice_detail(request, invoice_id):
    invoice = get_object_or_404(Invoice, id=invoice_id, customer=request.user.customer)
    return render(request, "invoice.html", {"invoice": invoice})

class ProfileForm(forms.ModelForm):
    class Meta:
        model = User
        fields = ["first_name", "last_name", "email"]
```

```python
def test_other_customers_invoice_is_404(client, alice, bobs_invoice):
    client.force_login(alice)
    assert client.get(f"/invoices/{bobs_invoice.pk}/").status_code == 404
```

MAKE THE MACHINE DO IT

One cross-customer test per model — boring, repetitive, mechanical. Exactly the assistant's strength.

In DRF: `DEFAULT_PERMISSION_CLASSES = [IsAuthenticated]`, globally.

OTEC
PRECISION FINISHING SOLUTIONS

CONFIGURATION — THE INSECURE WAY

> PROMPT
> The tests are slow and the deploy fails with DisallowedHost. Fix both.

```
# settings.py
DEBUG = True
SECRET_KEY = "django-insecure-8f3..."
ALLOWED_HOSTS = ["*"]
PASSWORD_HASHERS = [
    "django.contrib.auth.hashers.MD5PasswordHasher",
]
```

- Stack traces with settings, for anyone
- Signed data becomes forgeable
- Host header injection
- Password hashes cracked in no time

Both fixes are in the Django docs — the MD5 hasher **for tests**. The agent put it in the settings **everybody** uses.

OTEC
PRECISION FINISHING SOLUTIONS

CONFIGURATION — DJANGO TELLS YOU

```
$ python manage.py check --deploy

WARNINGS:
?: (security.W004) You have not set a value for the SECURE_HSTS_SECONDS setting.
?: (security.W009) Your SECRET_KEY has less than 50 characters ...
?: (security.W012) SESSION_COOKIE_SECURE is not set to True.
?: (security.W018) You should not have DEBUG set to True in deployment.
```

```
# django/contrib/auth/hashers.py (simplified)
class PBKDF2PasswordHasher(BasePasswordHasher):
    algorithm = "pbkdf2_sha256"
    iterations = 870_000            # 5.1 — raised with every release

# in the database: pbkdf2_sha256$870000$<salt>$<hash>
```

MAKE THE MACHINE DO IT

`python manage.py check --deploy --fail-level WARNING` in CI.

Test hasher in `settings/test.py` only. The agent does not touch `settings.py` — a human does.

ONE LINTER YOU ALREADY HAVE

bandit — static analysis for known-insecure Python patterns. In ruff as the `S` rules.

```
$ ruff check --select S

shop/views.py:12:9        S608  Possible SQL injection via string-based query construction
profiles/models.py:14:16  S308  Use of `mark_safe` may expose cross-site scripting
config/settings.py:9:14   S105  Possible hardcoded password assigned to: "SECRET_KEY"
tasks/runner.py:8:5       S602  `subprocess` call with `shell=True`
```

```
[tool.ruff.lint]
extend-select = ["S"]
```

> Everything a linter finds is **syntactic** — visible in the file.
> The missing ownership filter is not. No analyser knows who owns an invoice.

OTEC
PRECISION FINISHING SOLUTIONS

CONCLUSION — DJANGO IS SECURE BY DEFAULT

WHAT YOU GET FOR FREE

- Parameterised queries in the ORM
- Auto-escaping in templates
- CSRF middleware on every POST
- Password hashing, salted and slow
- `check --deploy` as a checklist

WHAT IT TAKES TO BREAK IT

- An f-string in `raw()`
- `|safe` / `mark_safe`
- `@csrf_exempt`
- One line in `PASSWORD_HASHERS`
- Not running it

> Making Django insecure is **one line**. Today it is often not us who types it — the assistant answers the question we asked.

OTEC
PRECISION FINISHING SOLUTIONS

ONE COMMAND. THREE LINES. TAKE HOME.

```
$ python manage.py check --deploy
```

```
security:
  script:
    - python manage.py check --deploy --fail-level WARNING
    - ruff check --select S
    - uv audit
```

> Your attention does not scale with the code you now produce. Your pipeline does.



THANK YOU

Questions?

gtan.eu · github.com/gaweng · LinkedIn · slides.gtan.eu/slides/django-insecure/

The best security feature in Django is the one you didn't turn off — and the one you didn't let a tool turn off for you.